SupportFlow

Visual Decision Tree Editor

Final design specification (v3)

A clear, interactive flowchart interface that replaces the Excel-based chatbot workflow. Built for non-technical users to visualize, edit and preview support conversations.

Status
READY TO BUILD

Version
3.0 - Final

Format
Landscape A4 - 8 pages

Contents

01 **Design System**
Tokens, grid, typography

02 **Data Model**
flow_data.json structure

03 **Node System**
All 6 states, ports, anatomy

04 **Editor View**
Toolbar, canvas, inspector

05 **Connections & Layout**
Rules, snap, auto-nudge

06 **Edge States & Wildcard**
Empty, error, auto-layout

07 **Preview Mode**
Chat simulation

Foundations

Single source of truth for colors, surfaces, typography and grid.

Semantic colors

Primary	Primary/S	Surface	Canvas	Background	Border	Text	Muted	Success	Warning	Danger
#2D6CDF	#1F4FB0	#FFFFFF	#FAFAFB	#F7F8FA	#E4E7EC	#0F172A	#64748B	#16A34A	#F59E0B	#DC2626

Typography

Display 22 / Bold

Heading 14 / Bold

Body 9.5 / Regular - the standard text size used in node cards.

Caption 7.5 / Regular - meta and helper text.

`Mono 7.5 - used for ids and code (q1, nextId, etc).`

Canvas - 14pt dot grid

Snap-to-grid is ON by default. Hold ALT to drag freely.

Spacing scale (pt)

4 8 12 16 24 32

Radius

r4 r6 r8 r12

Elevation

card / hover

flow_data.json

The runtime contract. Every UI element on the canvas maps 1:1 to this structure.

Schema

TypeScript reference - matches React state 1:1

```
type FlowData = {
  nodes: Node[];
  startId: string;
};

type Node = {
  id: string;             // 'q1'
  text: string;           // question
  position: {x:number;y:number};
  options: Option[];      // [] = end node
};

type Option = {
  label: string;          // button text
  nextId: string | null; // -> Node.id
};
```

Example - flow_data.json

Each option.nextId points to exactly one Node.id (or null = end).

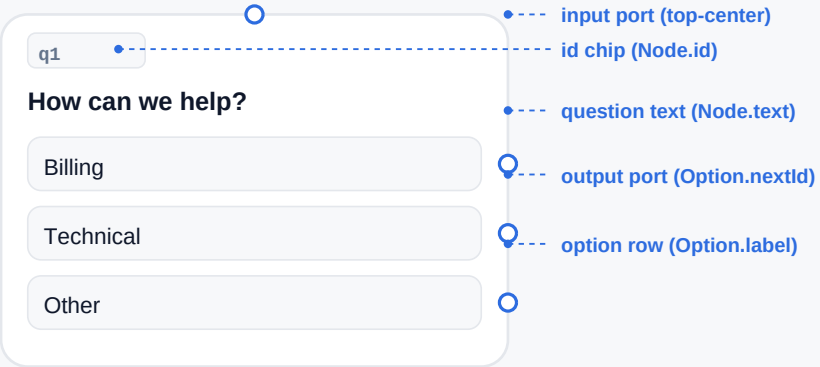
```
{
  "startId": "q1",
  "nodes": [
    {
      "id": "q1",
      "text": "How can we help?",
      "position": { "x": 120, "y": 80 },
      "options": [
        { "label": "Billing",  "nextId": "q2" },
        { "label": "Technical","nextId": "q3" }
      ]
    },
    {
      "id": "q3",
      "text": "Connect to agent?",
      "position": { "x": 480, "y": 220 },
      "options": [
        { "label": "Yes", "nextId": "end1" },
        { "label": "No",  "nextId": null }
      ]
    },
    {
      "id": "end1",
      "text": "Thanks! An agent will reach out.",
      "position": { "x": 760, "y": 220 },
      "options": []
    }
  ]
}
```

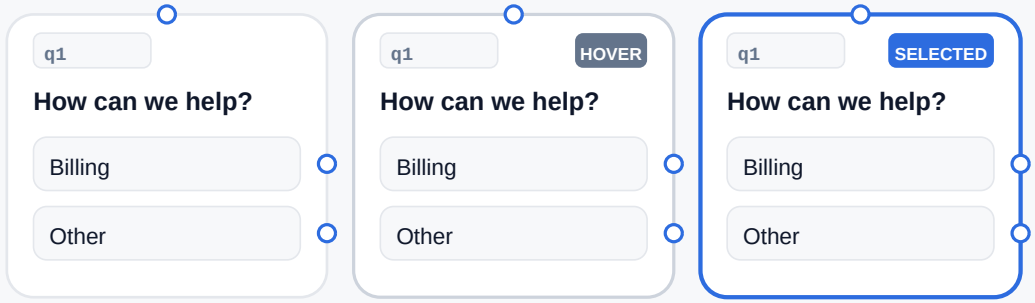
UI <-> Data mapping

Node card	**Question text**	**Card position**	**Option row**	**Output port**	**END card**
`Node`	`Node.text`	`Node.position`	`Option.label`	`Option.nextId -> Node.id`	`Node.options.length === 0`

Node anatomy & states

One component, six states. Identical typography, padding and port positions across all states.

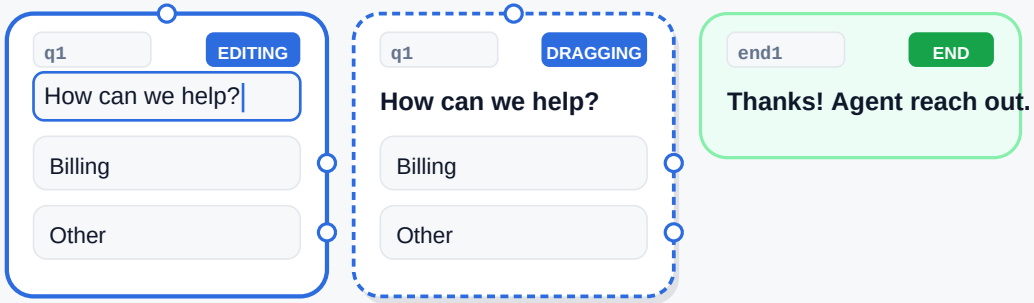




DEFAULT
Resting state

HOVER
Cursor over card

SELECTED
Clicked - inspector open



EDITING
Inline text edit

DRAGGING
Being moved

END
Terminal node (options=[])

Consistency rules

- Card width: 200pt fixed. Padding: 10pt. Radius: 10pt. Border: 1pt (1.6pt when selected/editing/dragging).
- Ports are always in the same place: input = top-center, output = right edge of each option row.
- END nodes share the exact card layout - only fill (#ECFDF5) and border (#86EFAC) differ. No options, no output ports.

+ Add
T Text
/ Link
A Auto
Z Zoom
? Help

- 100% + | fit

Flow: Customer Support > main

q1
How can we help?
Billing
Technical
Other

q2 HOVER
Billing question?
Invoice
Refund

end1 END
Thanks! Agent reach out.

q3 SELECTED
Connect to agent?
Yes
No

4 nodes - 3 connections - 0 issues

Saved just now

Inspector

PIN

Editing: q1

Node id
q1

Question text
How can we help?

Options

Billing x
-> q2

Technical x
-> q3

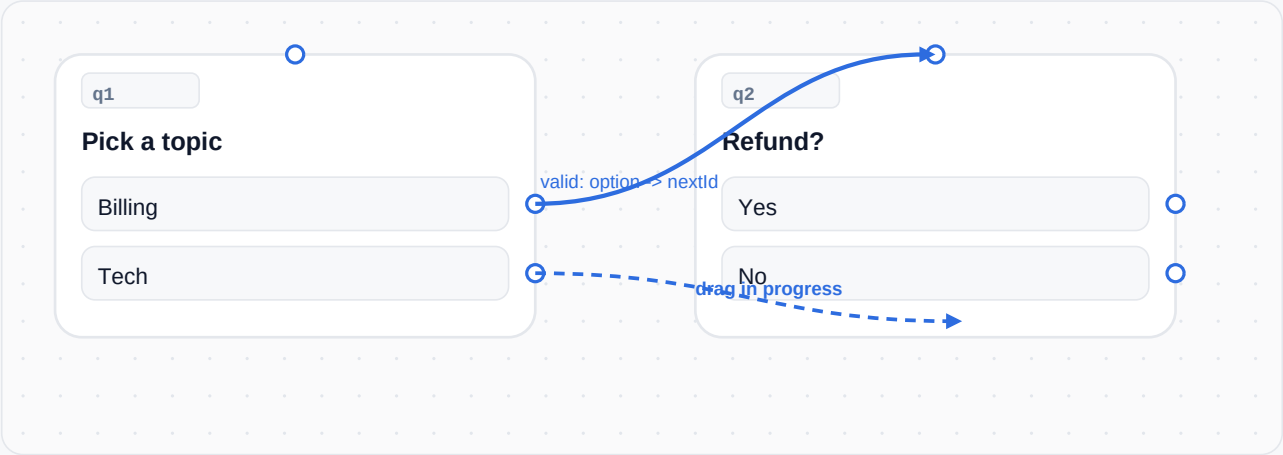
Other x
-> end1

+ Add option

Behavior
- Opens on node click
- Stays pinned (default ON)
- All edits sync in real-time

Connections

One option -> exactly one nextId. Directional. Always option.right -> node.top.



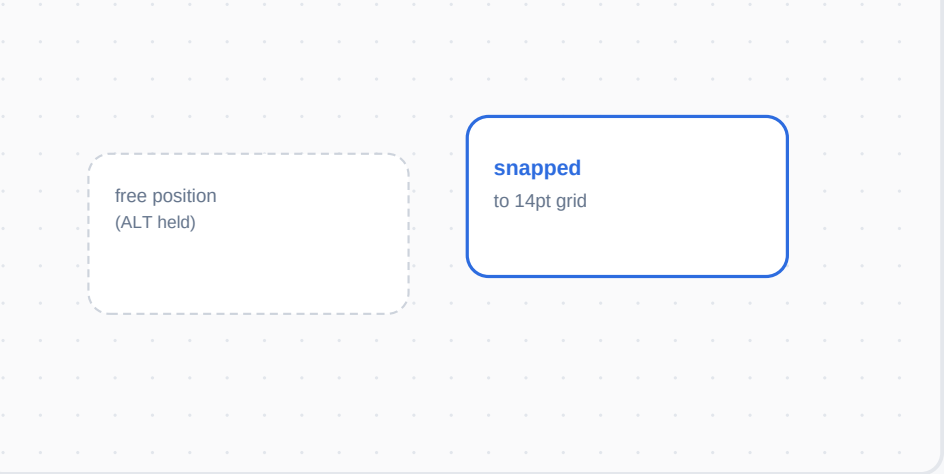
Connection rules

1. Each option has exactly one output port (right edge).
2. Each option connects to exactly one node via nextId.
3. Connections are directional (source -> target).
4. A node may have many incoming connections.
5. nextId = null renders an unconnected output -> warning chip.
6. Cycles are allowed (loops back to earlier nodes).
7. Connector style: 1.4pt curve, border-strong (#CDD3DC), primary (#2D6CDF) when selected/active, 6pt arrowhead.

Layout behavior

Snap-to-grid is the default. Auto-nudge prevents overlap.

Drag behavior



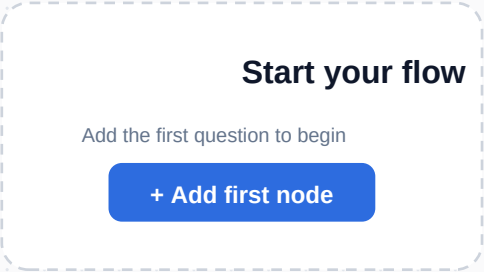
Spacing & overlap rules

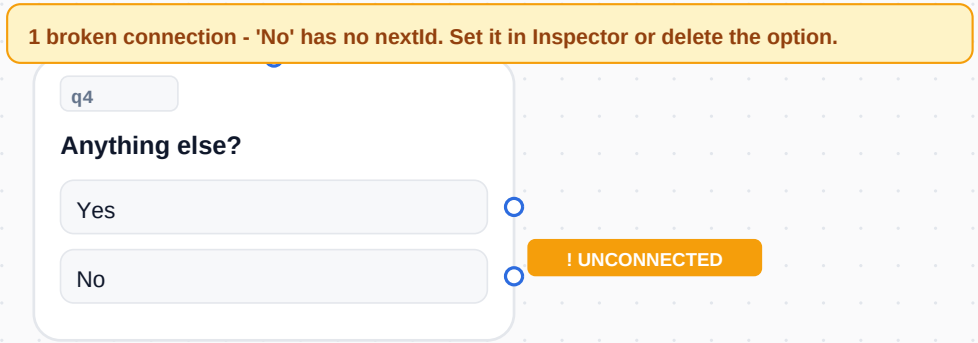
- Default drag: snap to 14pt grid. Hold ALT for free drag.
- Min horizontal gap between columns: 120pt.
- Min vertical gap between siblings: 40pt.
- On overlap: target node auto-nudges by one grid step (vertical first, horizontal as fallback).
- Zoom: 25% - 200%. Pan: hold SPACE + drag, or middle-mouse.
- Auto-Layout (toolbar 'A') re-tidies the whole flow into clean columns based on connection depth.
- Inspector behavior: opens on node click, stays pinned by default, all edits sync to canvas in real-time.

Edge states

Empty canvas, broken connections and the auto-layout wildcard.

Empty state



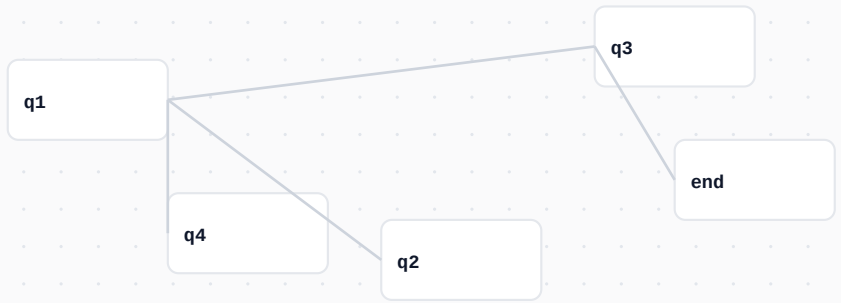


Broken-connection warning

Wildcard - Auto-Layout

One click tidies any messy flow into clean columns based on depth.

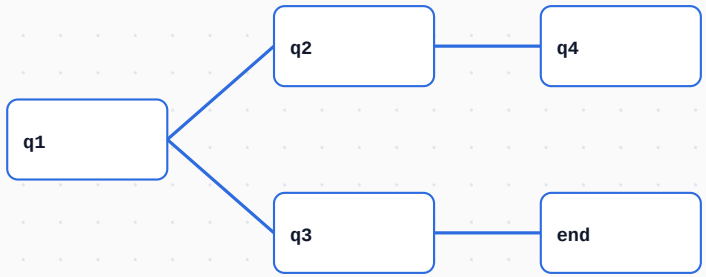
Before



>

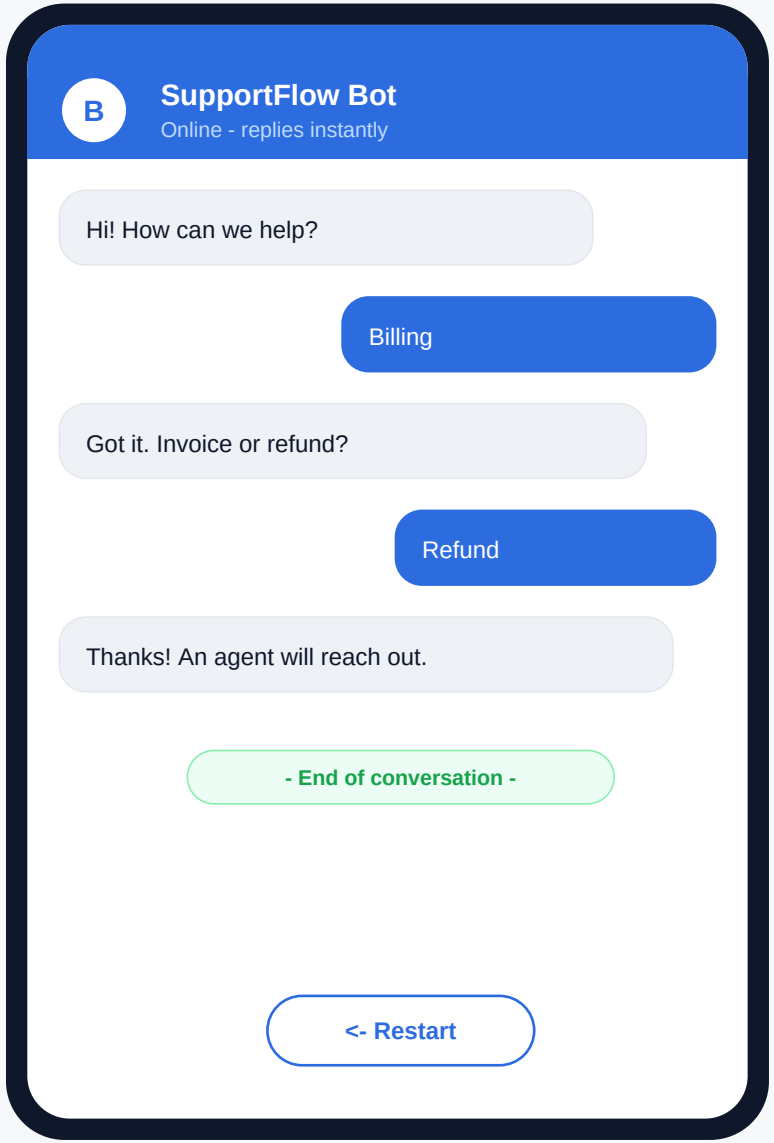
Auto

After



Preview Mode

Live chat simulation that walks the actual flow_data.json from startId.



Behavior

- Starts at flow_data.startId.
- Each user tap follows option.nextId.
- nextId === null OR options === [] -> END state.
- END shows the success chip + Restart button.
- Restart resets conversation to startId.
- All copy is read directly from Node.text / Option.label.
- No editing in Preview - read-only walkthrough.

Visual consistency

Same tokens are used in Editor and Preview.

- Connector	`1.4pt curve, #CDD3DC, 6pt arrow`
- Node radius	`10pt`
- Bubble radius	`10pt`
- Primary CTA	`#2D6CDF, white text`
- END accent	`#16A34A on #ECFDF5`